

CAIE Course Program

Assignment 6 - Build a RAG App | learn. build. operate

1. What you will build

Build a **RAG (Retrieval-Augmented Generation)** app using **LangChain** for the retrieval pipeline and **Streamlit** for the user interface. The user uploads a PDF in the browser. As soon as it is uploaded, your app reads the PDF, splits it into chunks, turns them into embeddings, and stores them in a **vector database**. Once that database is ready, the user can ask questions and get answers that are grounded in the contents of their PDF.

Difficulty: Medium · Tools: Python, LangChain, Streamlit, a vector store, OpenAI

2. What your app must do

- Show a **Streamlit** interface with a PDF file uploader.
- When a PDF is uploaded, load it, split it into chunks, create embeddings, and store them in a **vector database** (for example, FAISS or Chroma) so the database is ready to search.
- Give the user a query box to ask questions about the uploaded PDF.
- Use **LangChain** retrieval together with an OpenAI model to answer from the PDF content.
- Display the answer clearly in the app.
- Read your API key from an environment variable - never hard-code or commit it.
- Keep all your code in a single file named `app.py`.

Think it through

The vector database must be ready before the user can query it, so show a clear processing state while it builds and only enable the query box once it is done. Think about how you split the PDF into chunks, how you avoid rebuilding the database on every keystroke, and what happens if the PDF has no readable text.

3. Step-by-step setup

Follow these steps in order. Commands are shown for both macOS/Linux and Windows where they differ.

Step 1 - Create a project folder

```
mkdir rag-app  
cd rag-app
```

Step 2 - Create a virtual environment (venv)

A virtual environment keeps this project isolated from the rest of your system.

```
python -m venv venv  
# (on some systems use 'python3' instead of 'python')
```

Step 3 - Activate the virtual environment

```
# macOS / Linux:  
source venv/bin/activate  
  
# Windows (PowerShell):  
venv\Scripts\Activate.ps1  
  
# Windows (Command Prompt):  
venv\Scripts\activate.bat
```

When activated, you will see (venv) at the start of your terminal prompt.

Step 4 - Install the libraries

Install Streamlit, LangChain, the OpenAI integration, a PDF reader, and a vector store inside the activated venv. (You may swap the vector store for another one if you prefer.)

```
pip install streamlit langchain langchain-openai langchain-community pypdf  
faiss-cpu
```

Step 5 - Add your API key

Get an OpenAI key from platform.openai.com and set it as an environment variable before running.

```
# macOS / Linux:  
export OPENAI_API_KEY="your-openai-key"  
  
# Windows (PowerShell):  
$env:OPENAI_API_KEY="your-openai-key"
```

Step 6 - Run your app

Streamlit apps are started with `streamlit run`, not `python`.

```
streamlit run file_name.py
```

The app opens in your browser at <http://localhost:8501>.

Step 7 - Stop and deactivate when finished

Press `Ctrl + C` in the terminal to stop the app, then deactivate the venv.

```
deactivate
```

4. How it should behave

Your layout and wording can differ, but the flow should look roughly like this:

```
1. User uploads report.pdf
2. Building vector database... -> Ready
3. User asks: "What are the key findings?"
4. App replies with an answer grounded in the PDF
```

5. What to submit

- Push your code to **GitHub**.
- Share your GitHub repository link in **WhatsApp**.
- Post your screen-recorded video of the app working in the **community**.

Reminder: build the app yourself - the steps above are your setup roadmap, not the answer. Keep your API key out of GitHub. Good luck!